

DESENVOLVIMENTO DE UM SISTEMA DE APOIO À DECISÃO PARA O PLANEAMENTO DE ATIVIDADES EM OFICINAS DE MANUTENÇÃO AUTOMÓVEL



Departamento de Engenharia Mecânica

Instituto Superior de Engenharia do Porto

17 de julho de 2025

Candidato: Diogo Leme Abreu Coutinho, Nº 1211463, 1211463@isep.ipp.pt

Orientação científica: Pedro Rocha, pfr@isep.ipp.pt

Empresa: Instituto de Engenharia de Sistemas e Computadores, Tecnologia e Ciência

Supervisão: Ângelo Soares, angelo.v.soares@inesctec.pt



Departamento de Engenharia Mecânica

Instituto Superior de Engenharia do Porto

17 de julho de 2025

Agradecimentos

A concretização deste trabalho só foi possível graças ao apoio de várias pessoas e instituições a quem gostaria de expressar a minha sincera gratidão.

Em primeiro lugar, agradeço ao *INESC TEC* pela oportunidade de desenvolver este projeto em ambiente de estágio e aos meus orientadores, Pedro Rocha e Ângelo Soares pela orientação e disponibilidade demonstradas.

E um nível mais pessoal, uma palavra de profundo agradecimento aos meus pais e a toda a minha família, pelo apoio incondicional e incentivo constante que foram o meu pilar ao longo de todo este percurso.

Resumo

O presente trabalho aborda o problema da otimização do escalonamento de operações em oficinas de reparação automóvel, um setor caracterizado por uma elevada complexidade operacional e dominado por Pequenas e Médias Empresas (PMEs) que, frequentemente, não dispõem de acesso a ferramentas de gestão avançadas. O ambiente de uma oficina é marcado pela dualidade da procura entre manutenções agendadas e reparações urgentes e inesperadas e pela necessidade de alocar de forma eficiente um conjunto de recursos limitados, como mecânicos e postos de trabalho, a um leque variado de tarefas com restrições de precedência. Este desafio enquadra-se no problema de otimização combinatória conhecido como *Flexible Job Shop Scheduling Problem (FJSSP)*, cuja complexidade computacional torna a sua resolução manual ineficiente e propensa a erros.

A hipótese central deste projeto foi a de que é viável desenvolver um Sistema de Apoio à Decisão (SAD) eficaz e acessível, assente em técnicas de investigação operacional, para dar resposta a esta necessidade. Para tal, a principal contribuição deste trabalho consistiu na formulação de um modelo de Programação Linear Inteira Mista (MILP) que captura com rigor as características do problema, incluindo a flexibilidade na alocação de recursos.

A validação da solução proposta foi realizada através de uma metodologia experimental robusta e multifacetada. Primeiramente, desenvolveu-se uma arquitetura de testes em *Python*, composta por um gerador de instâncias parametrizado capaz de criar problemas com diferentes dimensões e níveis de complexidade e por um orquestrador que automatizou a execução e a recolha de resultados. Subsequentemente, conduziu-se uma análise de desempenho comparativa e aprofundada entre o solver comercial de referência, *Gurobi*, e duas alternativas *open-source* proeminentes, *HiGHS* e *CBC*. Esta análise, realizada sobre um conjunto diversificado de instâncias (de pequena, média e grande dimensão), permitiu avaliar cada solver com base em métricas de qualidade da solução (Gap Relativo ao Melhor Ótimo Conhecido) e eficiência computacional (Tempo de CPU)

Os resultados da análise experimental evidenciaram as principais vantagens e desvantagens da abordagem. A principal vantagem reside na capacidade de gerar planos de trabalho matematicamente otimizados. O solver *Gurobi* demonstrou uma superioridade inequívoca em todos os cenários, encontrando soluções de alta qualidade em tempo útil. A análise revelou também que o solver *HiGHS* constitui uma alternativa *open-source* notavelmente competente para problemas de média complexidade, superando claramente o CBC. A aplicação final do modelo num protótipo de SAD funcional, com uma interface gráfica desenvolvida em *Streamlit*, demonstrou o valor prático da ferramenta, traduzindo dados complexos em visualizações intuitivas (gráficos de *Gantt*) e fornecendo *insights* de gestão valiosos, como a quantificação da necessidade de horas de trabalho para além do turno normal.

Entre as desvantagens ou limitações, destaca-se o carácter determinístico do modelo, que não contempla a incerteza e a dinâmica de eventos inesperados. Adicionalmente, a análise comprovou que o desempenho das alternativas *open-source* se degrada significativamente em problemas de elevada complexidade, o que representa um desafio para uma implementação de baixo custo em operações de maior escala. Em suma, o trabalho validou com sucesso a hipótese formulada, resultando num protótipo funcional e numa análise de desempenho detalhada que serve de guia para a implementação de soluções de otimização no setor.

Palavras-Chave

Escalonamento de Produção, *Flexible Job Shop Scheduling Problem*, Otimização, Programação Linear Inteira Mista, Sistema de Apoio à Decisão, *Gurobi*, *HiGHS*, Manutenção Automóvel, Gestão de Operações.

Abstract

This work addresses the problem of optimizing operations scheduling in automotive repair shops, a sector characterized by high operational complexity and dominated by Small and Medium-sized Enterprises that often lack access to advanced management tools. The workshop environment is marked by a dual demand between scheduled maintenance and urgent, unexpected repairs, and by the need to efficiently allocate limited resources, such as mechanics and workstations, to a wide range of tasks with precedence constraints. This challenge fits within the combinatorial optimization problem known as the Flexible Job Shop Scheduling Problem (*FJSSP*), whose computational complexity makes manual resolution inefficient

The central hypothesis of this project was that it is feasible to develop an effective and accessible Decision Support System (*DSS*) based on operations research techniques to address this need. To this end, the main contribution of this work was the formulation of a Mixed-Integer Linear Programming (*MILP*) model that accurately captures the problem's characteristics, including the flexibility in resource allocation.

The proposed solution was validated through a robust and multifaceted experimental methodology. Firstly, a *Python-based* testing architecture was developed, comprising a parameterized instance generator capable of creating problems of different dimensions and complexity levels, and an orchestrator that automated the execution and collection of results. Subsequently, a thorough comparative performance analysis was conducted between the commercial benchmark solver, *Gurobi*, and two prominent open-source alternatives, *HiGHS* and *CBC*. This analysis, performed on a diverse set of instances (small, medium, and large), allowed for the evaluation of each solver based on solution quality metrics (Relative Gap to the Best-Known Optimum) and temporal efficiency (*CPU* Time).

The results of the experimental analysis highlighted the main advantages and disadvantages of the approach. The primary advantage lies in the ability to generate mathematically optimized work plans. The *Gurobi* solver demonstrated unequivocal superiority in all scenarios, finding high-quality solutions in a timely manner. The analysis also revealed that the *HiGHS* solver is a remarkably competent open-source alternative for medium-

complexity problems, clearly outperforming *CBC*. The final application of the model in a functional DSS prototype, with a graphical interface developed in Streamlit, demonstrated the tool's practical value by translating complex data into intuitive visualizations (*Gantt charts*) and providing valuable management insights, such as quantifying the need for overtime hours.

Among the disadvantages or limitations is the deterministic nature of the model, which does not account for the uncertainty and dynamics of unexpected events. Additionally, the analysis confirmed that the performance of the open-source alternatives degrades significantly on highly complex problems, which poses a challenge for a low-cost implementation in larger-scale operations. In summary, the work successfully validated the formulated hypothesis, resulting in a functional prototype and a detailed performance analysis that serves as a guide for implementing optimization solutions in the sector.

Keywords

Scheduling, Flexible Job Shop Scheduling Problem, Optimization, Mixed-Integer Linear Programming, Decision Support System, Gurobi, HiGHS, Automotive Repair, Operations Management.

Índice

AGRADECIMENTOS	I
RESUMO	III
ABSTRACT	V
ÍNDICE DE FIGURAS	XIII
ACRÓNIMOS.....	XV
1. INTRODUÇÃO	1
1.1. CONTEXTUALIZAÇÃO	1
1.2. OBJETIVOS	2
1.3. CALENDARIZAÇÃO	3
1.4. ORGANIZAÇÃO DO RELATÓRIO	3
2. REVISÃO DE LITERATURA E DESCRIÇÃO DO PROBLEMA.....	4
2.1. O PROBLEMA DE ESCALONAMENTO DE PRODUÇÃO.....	5
2.2. FLEXIBLE JOB SHOP SCHEDULING PROBLEM (FJSSP)	7
2.3. ABORDAGENS PARA A RESOLUÇÃO DE PROBLEMAS DE OTIMIZAÇÃO	8
2.4. SOLVERS DE OTIMIZAÇÃO MILP	9
3. METODOLOGIA E IMPLEMENTAÇÃO DA ABORDAGEM	13
3.1. FORMULAÇÃO MATEMÁTICA MODELO MILP	14
3.2. AMBIENTE E ARQUITETURA EXPERIMENTAL	17
3.1. DESENVOLVIMENTO DOS COMPONENTES DA ESTRUTURA DE TESTES	18
3.1. DESENHO DAS CAMPANHAS EXPERIMENTAIS	20
4. ANÁLISE DE DESEMPENHO E DISCUSSÃO DE RESULTADOS	24
4.1. INSTÂNCIAS DE NÍVEL SIMPLES	25
4.2. INSTÂNCIAS DE NÍVEL INTERMÉDIO	27
4.3. INSTÂNCIAS DE NÍVEL COMPLEXO	28
4.4. CONCLUSÃO DA ANÁLISE EXPERIMENTAL	30
5. DESENVOLVIMENTO DO SISTEMA DE APOIO À DECISÃO E APLICAÇÃO A CASO DE ESTUDO	32
5.1. CARACTERIZAÇÃO DO CASO DE ESTUDO.....	33
5.2. ARQUITETURA E IMPLEMENTAÇÃO DO SAD.....	34
5.3. INTERFACE E FLUXO DE UTILIZAÇÃO DO SAD	35
5.4. IMPACTO E CONTRIBUTOS DO SISTEMA DE APOIO À DECISÃO	40
6. CONCLUSÕES E TRABALHO FUTURO.....	42
REFERÊNCIAS	44

Índice de Figuras

Figura 1- Eficiência Temporal das Instâncias Simples	26
Figura 2 - Menu de entrada	36
Figura 3- Vista do Planeador com resultados.....	37
Figura 4 - Gantt por Operador.....	38
Figura 5- Gantt por Posto de Trabalho.....	38
Figura 6 - Tarefas que transitam para o dia seguinte	39
Figura 7 - Agenda operador	40
Figura 8 - Agenda posto de trabalho	40

Acrónimos

SAD	– Sistema de Apoio à Decisão
MILP	– Mixed-Integer Linear Programming (Programação Linear Inteira Mista)
FJSSP	– Flexible Job Shop Scheduling Problem
JSSP	– Job Shop Scheduling Problem
PMEs	– Pequenas e Médias Empresas
INESC TEC	– Instituto de Engenharia de Sistemas e Computadores, Tecnologia e Ciência
MOC	– Melhor Ótimo Conhecido (Best Known Optimum)
CBC	– COIN-OR Branch and Cut
HIGHS	– High-performance open-source software
GA	– Genetic Algorithms (Algoritmos Genéticos)
SA	– Simulated Annealing (Recozimento Simulado)
TS	– Tabu Search (Busca Tabu)
FIFO	– First-In, First-Out
SPT	– Shortest Processing Time
KPI	– Key Performance Indicator (Indicador Chave de Desempenho)

Error! No text of specified style in document.

1. INTRODUÇÃO

1.1. CONTEXTUALIZAÇÃO

O presente projeto insere-se no domínio da otimização de processos industriais, com um foco particular na gestão da produção em ambientes de serviços. O trabalho foi desenvolvido no âmbito de um estágio curricular no INESC TEC, uma instituição de referência na investigação e desenvolvimento tecnológico, cujo trabalho em investigação operacional motivou a aplicação de métodos quantitativos a um problema de elevada relevância para o tecido empresarial nacional. O desafio selecionado foi o escalonamento de operações em oficinas de reparação automóvel, um setor que representa um pilar vital da economia e da mobilidade, garantindo a operacionalidade e segurança de uma vasta frota de veículos. Em Portugal, esta realidade é particularmente expressiva, com um parque automóvel que, em 2020, ascendia a cerca de 7 milhões de veículos em circulação. A sustentação desta frota é assegurada por um tecido empresarial considerável, composto, no mesmo ano, por mais de 31 mil empresas no setor de comércio, manutenção e reparação.

O centro desta atividade reside nas oficinas, que enfrentam um ambiente operacional de crescente complexidade. O desafio central é a gestão eficiente do escalonamento de operações, que consiste na alocação de recursos limitados, como mecânicos e postos de trabalho, a um conjunto de tarefas ao longo do tempo. Uma estratégia de escalonamento eficaz traduz-se em benefícios diretos, como a minimização do tempo total de reparação, o cumprimento rigoroso dos prazos acordados com os clientes e a maximização da

produtividade. A complexidade desta tarefa é amplificada pela dualidade da procura: por um lado, as manutenções preventivas, que podem ser agendadas com antecedência; por outro, as manutenções corretivas, resultantes de avarias inesperadas, que introduzem um fator de incerteza constante no plano de trabalho diário.

Este problema é cientificamente enquadrado como o *Flexible Job Shop Scheduling Problem*, cuja complexidade *NP-Hard* o torna um desafio computacional significativo. Apesar da sua relevância crítica, as soluções de software comerciais existentes são, frequentemente, financeiramente inacessíveis para as pequenas e médias empresas que dominam o setor. Esta lacuna tecnológica justifica a necessidade de desenvolver soluções de otimização mais acessíveis e personalizáveis, sendo este o contexto que enquadra o presente trabalho.

1.2. OBJETIVOS

O presente trabalho aborda a otimização do planeamento de atividades em oficinas de manutenção automóvel, tendo como objetivo principal o desenvolvimento de um Sistema de Apoio à Decisão (*SAD*). A solução assenta na hipótese de que é possível, recorrendo a ferramentas de modelação matemática e a *solvers* de otimização, criar um sistema eficaz, acessível e adaptado às necessidades das PME's do setor.

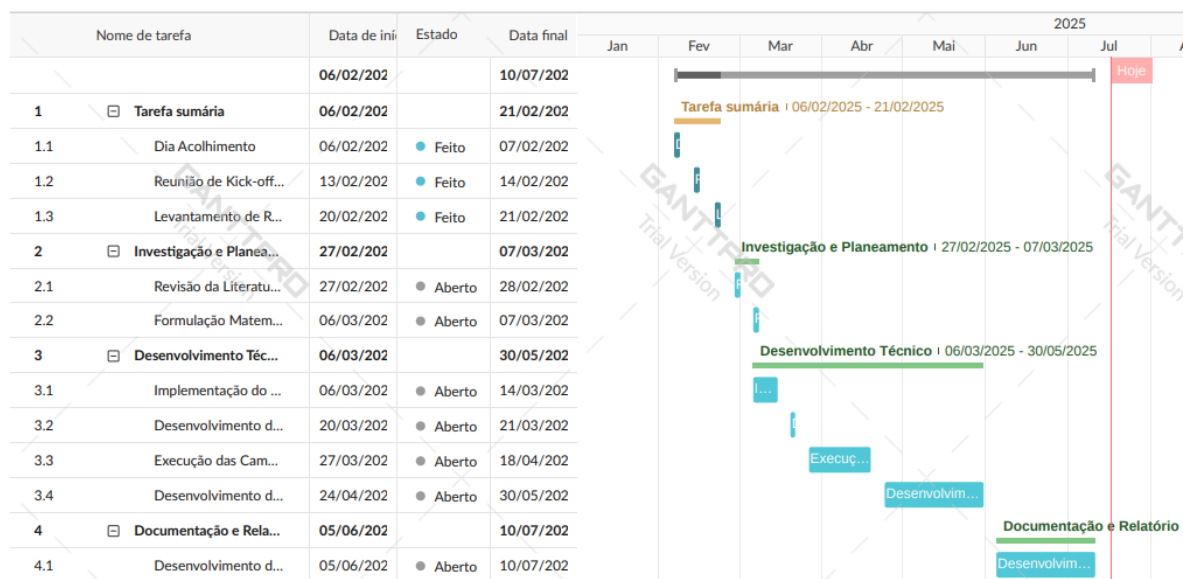
Para a concretização deste objetivo, foi delineado um plano de trabalho composto por várias tarefas sequenciais. O percurso metodológico iniciou-se com uma revisão da literatura e um estudo aprofundado do problema de escalonamento, de modo a fundamentar a abordagem teórica. Subsequentemente, procedeu-se à implementação de um modelo matemático de Programação Linear Inteira Mista (MILP), utilizando a linguagem *Python* e a biblioteca *PuLP* para garantir a compatibilidade com diferentes *solvers* de programação matemática.

Para validar a abordagem e selecionar o motor de otimização mais adequado, foi realizado o desenvolvimento de uma arquitetura de testes automatizada, que incluiu um gerador de instâncias parametrizado. Esta estrutura permitiu a realização de uma análise de resultados rigorosa, através de uma campanha experimental que contrapôs um solver comercial de referência (*Gurobi*) a alternativas *open-source* (*HiGHS*, *CBC*), utilizando métricas de qualidade da solução e eficiência temporal.

Finalmente, o culminar do trabalho consistiu no desenvolvimento de um protótipo do *SAD* que integra o modelo e o *solver* selecionado. Esta plataforma, desenvolvida com a ferramenta *Streamlit*, demonstra o seu valor prático ao ser aplicada na resolução de um caso de estudo representativo, cujo plano otimizado é apresentado de forma clara e funcional.

1.3. CALENDARIZAÇÃO

A prossecução dos objetivos delineados conduziu à calendarização de um conjunto de tarefas interdependentes, cuja execução se distribuiu ao longo do período de estágio. O projeto iniciou-se com uma fase de investigação e revisão bibliográfica, seguida pela modelação matemática do problema. A fase intermédia foi dedicada à implementação da arquitetura de testes, à execução das campanhas experimentais e à análise comparativa dos *solvers*. A fase final concentrou-se no desenvolvimento do protótipo do SAD, na sua aplicação a um caso de estudo e na redação do presente relatório.



1.4. ORGANIZAÇÃO DO RELATÓRIO

Este documento está estruturado em seis capítulos. O presente **capítulo 1** introduz o tema, os objetivos e a estrutura do trabalho. No **capítulo seguinte, o 2**, é apresentada a revisão de literatura, detalhando o problema de escalonamento, a sua complexidade e as principais abordagens de resolução. O **capítulo 3** dedica-se à exposição da metodologia utilizada para resolver o problema, desde a formulação do modelo matemático até ao funcionamento das

campanhas de análise dos *solvers*. No **capítulo 4**, são apresentados e discutidos os resultados obtidos nessa análise, culminando na seleção do *solver*. O **capítulo 5** descreve o desenvolvimento do protótipo do Sistema de Apoio à Decisão e a sua aplicação a um caso de estudo prático. Finalmente, no último **capítulo, o 6**, são reunidas as principais conclusões do trabalho, apresentadas as suas limitações e perspectivas futuros desenvolvimento

2. REVISÃO DE LITERATURA E DESCRIÇÃO DO PROBLEMA

Este capítulo tem como objetivo consolidar a fundamentação teórica que suporta o presente trabalho, abordando o problema de escalonamento de produção. O percurso expositivo inicia-se com uma descrição formal do problema de escalonamento (*scheduling*), enquadrando a sua importância estratégica na gestão de operações e apresentando os sistemas de classificação que permitem a sua caracterização. Partindo desta base, a análise foca-se no problema específico em estudo, o *Flexible Job Shop Scheduling Problem (FJSSP)*, detalhando as suas particularidades e a sua elevada complexidade computacional. Subsequentemente, realiza-se um levantamento das principais abordagens de resolução que têm sido propostas na literatura ao longo do tempo. Exploram-se tanto os métodos exatos, com especial ênfase na Programação Linear Inteira Mista (*MILP*) que constitui a base da nossa proposta, como os métodos aproximados, que surgiram como resposta à dificuldade de resolver problemas de grande dimensão. Por fim, a discussão aprofunda-se na análise dos *solvers* de otimização, as ferramentas de software que materializam estes algoritmos e que são o objeto central da análise experimental conduzida nos capítulos seguintes.

2.1. O PROBLEMA DE ESCALONAMENTO DE PRODUÇÃO (SCHEDULING)

2.1.1. DEFINIÇÃO E IMPORTÂNCIA ESTRATÉGICA

No âmbito da investigação operacional e da gestão industrial, um problema de escalonamento consiste na alocação de um conjunto de recursos limitados, como máquinas ou operadores, a um conjunto de tarefas ao longo do tempo, de modo a otimizar um ou mais objetivos de desempenho [1]. Esta alocação não é trivial, uma vez que está sujeita a um conjunto de restrições que podem incluir, entre outras, relações de precedência entre tarefas, janelas de tempo para a sua execução ou a possibilidade de interromper e retomar uma tarefa [2].

O escalonamento desempenha um papel estratégico na gestão da produção, influenciando diretamente a produtividade, a eficiência e a competitividade de um sistema produtivo. Um plano de escalonamento eficiente pode resultar no aumento da taxa de utilização dos recursos, na redução do tempo de ciclo dos produtos, no cumprimento rigoroso de prazos de entrega e, conseqüentemente, na minimização de custos operacionais e na melhoria da satisfação do cliente [3], [4].

2.1.2. EVOLUÇÃO HISTÓRICA DOS PROBLEMAS DE ESCALONAMENTO

O estudo formal dos problemas de escalonamento remonta à década de 1950, com contributos pioneiros que lançaram as bases teóricas da área. Destacam-se, entre os primeiros autores:

- Selmer M. Johnson (1954), considerado o marco inicial da teoria de *scheduling*, Johnson analisou o problema de *flow shop* com duas máquinas e apresentou um algoritmo ótimo para minimizar o *makespan*. O seu trabalho permitiu generalizar soluções também para três máquinas e tornou-se num modelo de análise académica.
- Jackson (1955–56) e Smith (1956) expandiram as ideias de Johnson aplicando-as a ambientes de *job shop* com duas máquinas e a problemas de uma única máquina com datas de entrega (*due dates*), respetivamente.

Já na década de 1970 aconteceram importantes desenvolvimentos teóricos como:

- Garey & Johnson (1976–79) estabeleceram formalmente a complexidade NPHard para muitas variantes do escalonamento, consolidando a base da teoria da *NPcompleteness*.
- Graham, Lawler, Lenstra & Rinnooy Kan (1979) introduziram a notação $\alpha|\beta|\gamma$, que se tornou o padrão para classificar formalmente os modelos de escalonamento. Esta última contribuição revelou-se fundamental para organizar o campo de estudo e permitir uma comparação rigorosa entre os diferentes problemas, como é detalhado de seguida.

2.1.3. SISTEMA DE CLASSIFICAÇÃO (NOTAÇÃO DE GRAHAM)

Dada a grande diversidade de problemas de escalonamento, tornou-se necessária uma notação formal para os classificar. A notação de três campos ($\alpha|\beta|\gamma$), introduzida por *Graham et al.* (1979), estabeleceu-se como o padrão na literatura académica para descrever de forma concisa as características de um problema de escalonamento [5].

O primeiro campo, α , descreve o ambiente das máquinas. Exemplos comuns incluem:

- Um ambiente com uma única máquina (*Single Machine*), onde todas as tarefas competem pelo mesmo recurso.
- Pm: Máquinas paralelas idênticas, onde qualquer uma das m máquinas pode processar qualquer tarefa com a mesma velocidade.
- Jm: Um ambiente Job Shop, onde cada tarefa (*job*) consiste numa sequência de operações, e cada operação tem uma máquina específica pré-designada para a sua execução.

O segundo campo, β , especifica as características das tarefas e as restrições do problema. Alguns exemplos são:

- *pmtn*: Indica que a preempção é permitida, ou seja, uma tarefa pode ser interrompida e retomada mais tarde.
- *prec*: Denota a existência de restrições de precedência genéricas entre as tarefas.
- r_j : Indica que cada tarefa j tem um tempo de chegada (release time) a partir do qual pode começar a ser processada.

O terceiro campo, γ , define a função objetivo a ser otimizada. As métricas mais comuns incluem:

- C_{max} : Minimização do *makespan*, que corresponde ao tempo de conclusão da última tarefa a sair do sistema. É uma medida da eficiência global.
- $\sum C_j$: Minimização da soma dos tempos de conclusão de todas as tarefas.
- $\sum w_j T_j$: Minimização da soma ponderada dos atrasos (*tardiness*), onde a *tardiness* de uma tarefa j , $T_j = \max(0, C_j - d_j)$, representa o atraso em relação à sua data de entrega estipulada d_j [4].

2.2. FLEXIBLE JOB SHOP SCHEDULING PROBLEM (FJSSP)

O *Job Shop Scheduling Problem* (JSSP) clássico é um dos problemas de escalonamento mais estudados e representa um desafio significativo. Nele, um conjunto de tarefas deve ser processado num conjunto de máquinas. Cada tarefa tem uma rota de produção rígida, ou seja, uma sequência predefinida de operações, onde cada operação deve ser processada numa máquina específica e dedicada [6]. A única decisão a ser tomada é a sequência em que as operações competem pelos mesmos recursos de máquina.

O *Flexible Job Shop Scheduling Problem* (FJSSP) é uma extensão mais realista e complexa do JSSP. A sua principal característica, a flexibilidade, reside no relaxamento da restrição de máquina dedicada. No FJSSP, cada operação de uma tarefa pode ser processada por qualquer máquina pertencente a um conjunto pré-definido de máquinas elegíveis [7]. Esta característica modela com maior fidelidade os cenários industriais modernos, onde é comum existirem máquinas ou postos de trabalho equivalentes capazes de realizar as mesmas operações.

2.3. ABORDAGENS PARA A RESOLUÇÃO DE PROBLEMAS DE OTIMIZAÇÃO

2.3.1. MÉTODOS EXATOS

Os métodos exatos caracterizam-se por serem algoritmos que não só encontram a solução ótima de um problema, como também provam formalmente que nenhuma solução superior existe, desde que lhes seja concedido tempo computacional suficiente. A abordagem exata mais proeminente para problemas de escalonamento é a Programação Matemática, nomeadamente a Programação Linear Inteira Mista (MILP).

A formulação de um problema como um modelo MILP permite capturar com rigor todas as suas restrições e objetivos. Um *solver* MILP explora o espaço de soluções de forma inteligente, utilizando algoritmos como o *Branch and Bound*. Este método resolve relaxações lineares do problema (ignorando temporariamente a integralidade das variáveis) para obter limites (bounds) sobre o valor da solução ótima, o que permite remover ramos da árvore de pesquisa que comprovadamente não contêm a solução ótima [9]. Uma extensão poderosa, o *Branch and Cut*, refina este processo ao adicionar dinamicamente restrições válidas (cortes) que reforçam a relaxação linear, aproximando-a do invólucro convexo das soluções inteiras e acelerando a convergência [10].

Apesar da sua robustez teórica, a natureza NP-Hard do FJSSP impõe severas limitações práticas aos métodos exatos. O tempo de execução destes algoritmos pode crescer exponencialmente com a dimensão do problema, tornando-os impraticáveis para instâncias de grande escala. Esta limitação é consistentemente documentada na literatura. Por exemplo, *Özver e Ünlüyurt* desenvolveram um modelo MILP para o FJSSP, mas verificaram que a sua aplicabilidade se restringia a problemas de pequena dimensão, levando-os a propor um algoritmo genérico para cenários mais complexos.

2.3.2. MÉTODOS APROXIMADOS: HEURÍSTICAS E META-HEURÍSTICAS

Para contornar as limitações dos métodos exatos, recorre-se frequentemente a métodos aproximados, que procuram encontrar soluções de alta qualidade em tempo computacionalmente aceitável, abdicando da garantia de ser ótimo.

As heurísticas, como as regras de despacho, são métodos construtivos que tomam decisões locais e rápidas. Exemplos incluem a regra *Shortest Processing Time* (SPT), que tende a minimizar o tempo médio de fluxo, ou a regra *First-In, First-Out* (FIFO) [11].

As meta-heurísticas são estratégias de busca de alto nível que guiam heurísticas subjacentes para explorar o espaço de soluções de forma mais eficaz, equilibrando a exploração de novas regiões (*diversification*) com a refinação de boas soluções já encontradas (*intensification*). Para o FJSSP, várias meta-heurísticas têm sido aplicadas com sucesso, incluindo *Genetic Algorithms* (GA), *Simulated Annealing* (SA) e *Tabu Search* (TS) [7], [11].

2.4. SOLVERS DE OTIMIZAÇÃO MILP

2.4.1. DEFINIÇÃO DE UM SOLVER DE OTIMIZAÇÃO

Um *solver* de otimização é um *software* altamente especializado que implementa os algoritmos numéricos, como os descritos na secção 2.3.1, para resolver modelos de programação matemática. Na prática, um *solver* MILP recebe como entrada uma formulação do problema (variáveis, restrições e função objetivo) e devolve a melhor solução encontrada, juntamente com um certificado da sua qualidade (o gap de otimalidade) [9]. Estes motores de otimização são a peça central de inúmeros sistemas de apoio à decisão na indústria, sendo utilizados para otimizar desde cadeias de abastecimento a planos de produção.

2.4.2. PERFIL DOS SOLVERS EM ESTUDO

Para a análise experimental deste trabalho, foram selecionados três *solvers* MILP: um solver comercial de referência e duas alternativas *open-source*.

O *Gurobi*, que é considerado o padrão de excelência no mercado comercial, foi desenvolvido por pioneiros da otimização e é reconhecido pela sua velocidade e robustez na resolução de problemas LP, MILP e outros [12]. Implementa os algoritmos mais avançados de *branch-and-cut*, heurísticas primais e processamento paralelo, sendo a escolha preferencial em muitas aplicações industriais críticas. Apesar de disponibilizar licenças gratuitas para fins acadêmicos, representa um investimento considerável para o meio empresarial.

O *CBC* (*COIN-OR Branch and Cut*): É um dos projetos mais maduros e utilizados da fundação COIN-OR (*Computational Infrastructure for Operations Research*). Sendo *open-source*, oferece grande flexibilidade e é uma base sólida para muitas aplicações acadêmicas e comerciais que requerem um solver MILP sem custos de licença. Embora o seu desempenho em instâncias de grande escala seja geralmente inferior ao dos concorrentes comerciais, a sua fiabilidade e acessibilidade tornam-no uma ferramenta fundamental na comunidade [13].

O *HiGHS* (High-performance open-source software) é um projeto *open-source* mais recente, focado em competir diretamente com os *solvers* comerciais em termos de desempenho. Desenvolvido na Universidade de Edimburgo, o HiGHS tem vindo a ganhar notoriedade pela sua performance excecional, particularmente na resolução de problemas de Programação Linear (LP) de grande escala, e as suas capacidades em MILP estão em rápido desenvolvimento. A sua licença permissiva (MIT) facilita a sua integração em *software* comercial [14].

2.4.3. ANÁLISE COMPARATIVA

A escolha de um *solver* é uma decisão técnica com impacto direto no sucesso de um projeto de otimização. A literatura acadêmica fornece uma base para estabelecer uma hipótese de trabalho sobre o desempenho relativo dos *solvers* selecionados.

Benchmarks, como os realizados por Hans Mittelmann, consistentemente posicionam *solvers* comerciais como o *Gurobi* na liderança em termos de velocidade e robustez para problemas MILP [20]. Estudos comparativos em domínios específicos, como o planeamento de sistemas de energia, corroboram esta tendência. Por exemplo, Ye et al. (2022) e Hall et al. (2022) mostram que, enquanto o HiGHS apresenta um desempenho notável e competitivo em problemas de pequena e média escala, aproximando-se dos comerciais, o *Gurobi* mantém uma vantagem substancial (reportada como sendo entre 60 a 100 vezes mais rápido) em instâncias de grande dimensão e complexidade [21], [22]. Nestes mesmos cenários, *solvers* como o CBC e o GLPK, embora fundamentais para a comunidade *open-source*, não se mostram competitivos em termos de tempo de resolução. Relatos em comunidades técnicas sugerem que, para problemas de escalonamento, o *Gurobi* pode superar o CBC por uma margem de 15 a 100 vezes em termos de tempo computacional e encontrar a solução ótima [23].

Estas observações permitem formular a hipótese de que, para a análise experimental a ser conduzida, se espera uma clara superioridade de desempenho do *Gurobi*, seguido pelo HiGHS, com o CBC e o GLPK a apresentarem tempos de resolução significativamente mais elevados, especialmente nas instâncias mais complexas.

3. METODOLOGIA E IMPLEMENTAÇÃO DA ABORDAGEM

Este capítulo é dedicado à apresentação detalhada da metodologia central desenvolvida para resolver o problema de escalonamento em estudo. A abordagem adotada consiste na formulação de um modelo de Programação Linear Inteira Mista (MILP), uma técnica de otimização poderosa e amplamente utilizada em investigação operacional.

A escolha desta metodologia fundamenta-se na sua capacidade para representar com precisão e rigor as complexas relações de precedência, atribuição e sequenciamento que caracterizam o *Flexible Job Shop Scheduling Problem* (FJSSP). Para uma exposição clara e sistemática, a apresentação do modelo seguir-se-á a estrutura convencional, iniciando-se com a definição dos conjuntos, parâmetros e variáveis de decisão. Subsequentemente, serão detalhadas a função objetivo, que formaliza o critério de otimização, e o conjunto de restrições que governam o comportamento do sistema, garantindo a validade e a viabilidade das soluções geradas.

Esta formulação matemática constitui a base sobre a qual toda a análise experimental e o desenvolvimento do protótipo, descritos nos capítulos seguintes, foram construídos.

3.1. FORMULAÇÃO MATEMÁTICA DO MODELO MILP

Para que um problema de otimização possa ser resolvido computacionalmente, é imperativo traduzir a sua descrição conceptual numa formulação matemática rigorosa. A abordagem seleccionada para este trabalho foi a Programação Linear Inteira Mista (MILP), uma técnica poderosa capaz de capturar com precisão a complexidade do problema em estudo.

O problema de escalonamento da oficina é modelado como um *Flexible Job Shop Scheduling Problem*. Esta formulação é particularmente adequada porque lida com as duas decisões centrais do planeador:

1. Problema de Atribuição: Decidir que recurso (aqui designado genericamente como "máquina") irá realizar cada operação.
2. Problema de Sequenciamento: Definir a ordem pela qual as operações competirão e serão executadas em cada recurso.

A seguir, apresenta-se o modelo matemático desenvolvido, que estrutura o problema e serve de base a toda a análise experimental e ao desenvolvimento do protótipo

3.1.1. ÍNDICES E CONJUNTOS

$j, l \in J$: Índice e conjunto das tarefas (viaturas)

$i, p \in O_j$: Índice e conjunto das operações de uma dada tarefa j .

$k \in M$: Índice e conjunto das máquinas (recursos genéricos).

$M_{j_i} \subseteq M$: Subconjunto de máquinas elegíveis para realizar a operação i da tarefa j .

3.1.2. PARÂMETROS

p_{jik} : Tempo de processamento da operação i da tarefa j na máquina k .

r_j : Instante de chegada da tarefa j .

L : Constante positiva de valor elevado (Big-M).

3.1.3. VARIÁVEIS DE DECISÃO

$s_{ji} \geq 0$: Instante de início da operação i da tarefa j .

$C_{max} \geq 0$: *Makespan* do plano de trabalho.

$x_{jik} \in \{0,1\}$: Variável binária de atribuição (1 se a máquina k faz a operação (j,i)).

$y_{jilpk} \in \{0,1\}$: Variável binária de sequenciamento (1 se a operação (j,i) precede a (l,p) na máquina k).

3.1.4. FUNÇÃO OBJETIVO

O objetivo do modelo é minimizar o tempo total necessário para concluir todas as tarefas, ou seja, minimizar o *makespan*.

Minimizar C_{max}

3.1.5. RESTRIÇÕES

$$\sum_{k \in M_{ji}} j_{ik} = 1 \quad \forall j \in J, \forall i \in O_j \quad (1)$$

$$S_{j,i+1} \geq s_{ji} + \sum_{k \in M_{ji}} p_{jik} \cdot x_{jik} \quad \forall j \in J, \forall i \in O_j : i < |O_j| \quad (2)$$

$$s_{j,1} \geq r_j \quad \forall j \in J \quad (3)$$

$$S_{lp} \geq s_{ji} + \sum_{k \in M_{ji}} p_{jik'} \cdot x_{jik'} - L \cdot (1 - y_{jilpk}) - L \cdot (2 - x_{jik} - x_{lpk}) \quad (4)$$

$$S_{ji} \geq s_{lp} + \sum_{k \in M_{ji}} p_{jik'} \cdot x_{jik'} - L \cdot (1 - y_{jilpk}) - L \cdot (2 - x_{jik} - x_{lpk}) \quad (5)$$

$$C_{max} \geq s_{ji} + \sum_{k \in M_{ji}} p_{jik} \cdot x_{jik} \quad \forall j \in J, \forall i \in O_j$$

$$s_{ji} \geq 0, C_{max} \geq 0$$

$$x_{jik} \in \{0,1\}, y_{jilpk} \in \{0,1\}$$

3.1.6. EXPLICAÇÃO DAS RESTRIÇÕES

O conjunto de restrições apresentado constitui a estrutura lógica do modelo, assegurando que qualquer solução gerada é operacionalmente viável e respeita todas as regras do problema. Primeiramente, a restrição de atribuição **(1)** garante que cada operação é alocada a uma, e apenas uma, máquina de entre as que são elegíveis, governando a decisão de alocação de recursos. De seguida, são impostas as regras temporais: a restrição de precedência **(2)** força a ordem tecnológica correta dentro de cada tarefa, enquanto a restrição de tempo de chegada **(3)** ancora o plano no tempo real, impedindo que o trabalho comece antes da chegada da viatura à oficina. O núcleo do modelo reside nas restrições disjuntivas **(4)** e **(5)**, que gerem a competição pelos recursos. Através da formulação "*Big-M*" e da variável de decisão binária y , este par de equações impede a sobreposição temporal, garantindo que duas operações atribuídas à mesma máquina nunca são processadas em simultâneo ao forçar uma sequência entre elas. Finalmente, a restrição **(6)** define o *makespan* como sendo igual ou superior ao instante de conclusão de todas as operações, ligando a estrutura do problema à função objetivo. Em conjunto, e com as restrições de domínio que definem a natureza das variáveis, estas equações traduzem o complexo problema de escalonamento da oficina numa estrutura matemática rigorosa, pronta a ser resolvida por um solver de otimização.

3.2. AMBIENTE E ARQUITETURA EXPERIMENTAL

Todos os testes computacionais foram realizados num PC com um processador Intel Core i7-12700H a 2.70 GHz, 16 GB DDR4 de memória RAM DDR4 e um disco de armazenamento de estado sólido (SSD) NVMe.

A linguagem de programação de eleição para o desenvolvimento de todas as ferramentas foi o *Python*, na sua versão 3.13, devido à sua vasta biblioteca de pacotes para computação científica e à sua sintaxe clara, que facilita o desenvolvimento e a manutenção de código complexo.

Dentro do ambiente Python, a biblioteca PuLP desempenhou um papel central. Foi utilizada para a formulação do modelo matemático do FJSSP, uma vez que esta funciona como uma interface de alto nível que permite modelar o problema de forma agnóstica em relação ao solver subjacente. Esta capacidade de abstração foi fundamental para o presente trabalho, pois permitiu executar o mesmo modelo nos três solvers em estudo sem necessidade de reescrita ou adaptação do código para cada um deles. Para a manipulação e posterior análise dos dados gerados, a biblioteca Pandas foi selecionada pela sua reconhecida eficiência e flexibilidade no tratamento de estruturas de dados tabulares.

O núcleo da análise comparativa reside nos três *solvers* de otimização selecionados, que representam diferentes paradigmas do mercado: o comercial de referência e três alternativas *open-source*. O *solver Gurobi* foi utilizado através da sua interface de linha de comandos, invocada pelo PuLP. De forma análoga, as alternativas *open-source CBC* e *HiGHS* foram integradas através das respetivas interfaces que o PuLP disponibiliza, garantindo um método de chamada consistente entre todos os concorrentes. A especificação das versões exatas de cada *solver*, detalhada na Tabela 3.1, é um passo crucial para garantir a rastreabilidade e a reprodutibilidade dos resultados obtidos.

Solver	Versão	Tipo de Licença
Gurobi Optimizer	12.0.1	Licença Académica
CBC (COIN-OR)	2.10.12	Eclipse Public License 2.0 (Open-Source)
HiGHS	1.9	MIT License (Open-Source)

Tabela 1 - Solvers de otimização e respetivas versões utilizadas na análise experimental

3.3. DESENVOLVIMENTO DOS COMPONENTES E ESTRUTURA DE TESTES

Para a condução de uma análise experimental rigorosa e sistemática, foi desenvolvida uma arquitetura de testes em *Python*, composta por um conjunto de componentes de software interdependentes. Esta abordagem modular permitiu não só a automatização completa do fluxo de trabalho, desde a geração dos problemas até à recolha dos resultados, mas também garantiu um controlo preciso sobre todas as variáveis da experiência. A seguir, cada um destes componentes é descrito em detalhe

3.3.1. GERADOR DE INSTÂNCIAS

A validade de uma análise comparativa de *solvers* depende intrinsecamente da qualidade e diversidade do conjunto de problemas utilizado. Confiar apenas em instâncias da literatura ou em dados de um único caso real poderia introduzir vieses ou não explorar suficientemente os limites de cada *solver*. Para mitigar este risco, foi desenvolvido um gerador de instâncias.

A lógica do gerador assenta na parametrização de seis parâmetros que definem a estrutura e a complexidade de um problema de escalonamento. Ao variar estes parâmetros dentro de intervalos predefinidos, é possível criar cenários com características muito distintas. Os parâmetros utilizados são: o número de tarefas (N), o número de recursos/máquinas (M), o número de operações por tarefa (O), a gama de tempos de processamento (P), o nível de flexibilidade (Flex) – que representa a percentagem média de máquinas elegíveis para cada operação – e a gama de tempos de chegada das tarefas (R).

Com base nestes parâmetros, foram definidas três classes de instâncias – Pequena, Média e Grande – de forma a avaliar os *solvers* em diferentes regimes de dificuldade computacional. A Tabela 3 detalha os intervalos de valores para cada classe. As instâncias de classe Pequena foram desenhadas para serem, em teoria, resolúveis até à otimalidade num curto espaço de tempo, testando a velocidade base dos *solvers*. As instâncias de classe Média representam um desafio computacional mais significativo, típico de problemas de média dimensão. Finalmente, as instâncias de classe Grande foram concebidas para testar a robustez e a escalabilidade dos *solvers*, sendo expectável que estes atinjam o limite de tempo imposto, o que permite avaliar a qualidade das suas heurísticas internas na procura de soluções viáveis.

Parâmetro	Classe Pequena	Classe Média	Classe Grande
Nº de Tarefas (N)	[5-10]	[11-20]	[21-35]
Nº de Máquinas (M)	[3-5]	[6-8]	[9-12]
Nº de Operações (O)	[2-4]	[4-6]	[6-8]
Gama de Tempos (P)	[5-20]	[10-40]	[15-60]
Flexibilidade (Flex)	[75%-100%]	[50%-75%]	[25%-50%]

Tabela 2 - Definição das classes de instâncias geradas

3.3.2. FLUXO DE TRABALHO PARA AUTOMAÇÃO EXPERIMENTAL

Para garantir a execução rigorosa e a reprodutibilidade da análise comparativa entre os *solvers*, foi desenvolvida uma arquitetura de software em *Python* para automatizar todo o fluxo de trabalho experimental. Esta estrutura é responsável por três funções essenciais e sequenciais.

Primeiramente, um orquestrador experimental gere a execução sistemática dos testes, submetendo cada instância do problema a cada um dos *solvers* em estudo, sob as mesmas condições e com um limite de tempo pré-definido. Este componente registra não só os resultados primários da otimização, mas também os *logs* detalhados de cada processo.

Numa segunda fase, uma pipeline de pós-processamento transforma os dados brutos gerados. Esta pipeline consolida os resultados, extrai métricas de desempenho avançadas a partir dos *logs* (como o tempo para encontrar a primeira solução) e calcula indicadores comparativos essenciais, como o Melhor Ótimo Conhecido (*MOC*) para cada instância.

Finalmente, a arquitetura culmina na geração automática de análises e visualizações. Com base no conjunto de dados consolidado, são produzidos todos os gráficos, tabelas e perfis de desempenho necessários para a discussão de resultados. Esta abordagem automatizada foi fundamental para assegurar a consistência da análise, minimizando a intervenção manual e permitindo uma avaliação aprofundada e objetiva do desempenho de cada solver.

3.4. DESENHO DAS CAMPANHAS EXPERIMENTAIS

Com a arquitetura de testes implementada, o passo seguinte consistiu no desenho de um conjunto de campanhas experimentais com o propósito de avaliar e comparar rigorosamente o desempenho dos solvers selecionados. A questão central a que esta análise se propõe responder não é simplesmente determinar qual o solver superior, mas sim quantificar a magnitude dessa diferença de desempenho e aferir a viabilidade das alternativas open-source (*HiGHS* e *CBC*) como motores de um Sistema de Apoio à Decisão de baixo custo. Pretende-se, portanto, compreender quão perto estas ferramentas se encontram do *benchmark* estabelecido pelo *Gurobi*, em diferentes cenários de complexidade.

Para este efeito, a análise experimental foi focada na análise de desempenho geral para obter uma visão abrangente do comportamento de cada solver. Para tal, foi utilizado um conjunto de teste composto por 5 instâncias de cada uma das três classes de dificuldade (Pequena, Média e Complexa), totalizando 15 problemas distintos. Cada uma destas instâncias foi resolvida pelos três solvers em análise, tendo sido imposto um limite de tempo de execução de 3600 segundos (1 hora) a cada processo. Este limite temporal foi considerado suficiente para permitir a resolução completa das instâncias mais simples e, simultaneamente, para testar a capacidade dos solvers em encontrar soluções de alta qualidade nos problemas mais exigentes antes de o atingirem.

A avaliação comparativa do desempenho dos solvers foi realizada com base num conjunto de métricas quantitativas, que se dividem em três categorias principais:

1. **Qualidade da Solução:** A métrica primária é o Gap Relativo ao *MOC*, que mede a diferença percentual entre a solução de um solver e a melhor solução encontrada entre todos para essa mesma instância. A Taxa de Sucesso, ou a percentagem de vezes que um solver atinge o *MOC*, é também utilizada para medir a sua eficácia.
2. **Eficiência Temporal:** Avaliada através do Tempo de *CPU* total de execução. A análise da distribuição desta métrica permite comparar a velocidade e a consistência de cada solver.
3. **Agilidade e Robustez:** A agilidade é aferida através da análise da qualidade da primeira solução encontrada. A robustez geral é sintetizada através de perfis de

desempenho, um método gráfico que compara a eficiência e a qualidade das soluções de todos os solvers num único diagrama.

A aplicação desta metodologia rigorosa permite gerar um conjunto de dados rico e multifacetado, cuja análise detalhada, apresentada no capítulo seguinte, irá fundamentar as conclusões sobre a seleção do motor de otimização mais adequado para o problema em estudo.

4. ANÁLISE DE DESEMPENHO E DISCUSSÃO DE RESULTADOS

Este capítulo apresenta e discute os resultados obtidos através das campanhas experimentais cuja metodologia foi detalhada no Capítulo 3. A análise que se segue tem um duplo objetivo. Em primeiro lugar, realizar uma avaliação comparativa aprofundada do desempenho dos *solvers* *Gurobi*, *HiGHS* e *CBC*, focando-se na sua eficácia, eficiência e robustez. Em segundo lugar, e de forma crucial, utilizar estas conclusões para fundamentar a seleção do *solver* mais adequado a ser integrado no Sistema de Apoio à Decisão (SAD), o qual constitui a aplicação prática final deste trabalho.

A análise procura responder à questão central de aferir a viabilidade das alternativas open-source em relação ao *benchmark* comercial estabelecido pelo *Gurobi*. Para tal, a discussão está estruturada de forma a seguir uma ordem crescente de complexidade, iniciando-se pelas instâncias de nível Simples, progredindo para as de nível Intermédio e terminando nas de nível Complexo. Esta abordagem narrativa permite construir uma análise mais intuitiva: a avaliação dos cenários mais simples estabelece uma base de comparação, a transição para os problemas de dificuldade intermédia expõe as primeiras divergências de desempenho e, finalmente, a análise dos casos mais exigentes serve para testar os limites da robustez e da escalabilidade de cada ferramenta.

Para cada nível, serão avaliadas métricas de qualidade da solução, eficiência temporal e robustez, com recurso às visualizações geradas pela pipeline de análise, de modo a construir uma conclusão informada e justificada sobre o motor de otimização ideal para a plataforma a desenvolver.

4.1. INSTÂNCIAS DE NÍVEL SIMPLES

A análise comparativa do desempenho dos *solvers* inicia-se pelas instâncias de nível simples. Nestes cenários, caracterizados por uma complexidade computacional reduzida, a qualidade da solução não se constitui como o principal fator de diferenciação entre os *solvers*. O foco primordial recai sobre a eficiência computacional, ou seja, a velocidade com que cada solver consegue encontrar e provar a otimalidade da solução.

A Tabela seguinte apresenta os resultados detalhados para este conjunto de problemas.

Instância	Solver	Makespan Final	Tempo CPU (s)	Gap Relativo ao MOC (%)
InstS_01	Gurobi	93	0.12	0.00
	HiGHS	93	0.45	0.00
	CBC	93	0.45	0.00
InstS_02	Gurobi	56	0.11	0.00
	HiGHS	56	0.28	0.00
	CBC	56	0.29	0.00
InstS_03	Gurobi	73	0.27	0.00
	HiGHS	73	7.68	0.00
	CBC	73	34.91	0.00
InstS_04	Gurobi	77	1.9	0.00
	HiGHS	77	355.54	0.00
	CBC	79	3600*	2.60
InstS_05	Gurobi	76	0.19	0.00
	HiGHS	76	1.98	0.00
	CBC	76	0.68	0.00

Tabela 4 - Resultados detalhados para as instâncias de nível Simples

Conforme antecipado, os resultados demonstram que, para problemas de baixa complexidade, todos os *solvers* são altamente eficazes na obtenção da solução ótima. Tanto o *Gurobi* quanto o *HiGHS* atingiram o *MOC* em 100% dos casos. O *CBC*, por sua vez, demonstrou uma taxa de sucesso ligeiramente inferior, falhando em encontrar o *MOC* em apenas uma das instâncias testadas.

Métrica	Gurobi	HiGHS	CBC
Makespan Médio	75	75	75.4
Tempo CPU Médio (s)	0.5	73.2	727.3
Taxa de Sucesso -Atingiu MOC (%)	100	100	80

Tabela 5 - Desempenho médio dos solvers nas instâncias de nível Simples

O verdadeiro diferenciador neste nível de complexidade é, sem dúvida, a eficiência computacional. A análise do tempo de CPU médio revela uma disparidade significativa entre os *solvers*. O *Gurobi* destaca-se por resolver todos os problemas com alta eficiência computacional, tendo um tempo médio de 0.5s nas instancias de nível mais simples.

O *HiGHS*, embora também rápido em comparação com o CBC, é consideravelmente mais lento que o *Gurobi*, registrando um tempo médio de 73.2 segundos. O seu desempenho é influenciado por uma maior variabilidade no tempo de resolução, como se observa em algumas instâncias individuais.

Por fim, o CBC revela-se o menos eficiente neste patamar, com um tempo médio de 727.3 segundos. Esta média é substancialmente impactada pela sua incapacidade de resolver a instância *Inst_S_04* dentro do limite de tempo imposto de 3600 segundos, o que evidencia as suas limitações mesmo em problemas de menor dimensão.

A visualização gráfica dessas disparidades de tempo de resolução pode ser apresentada para reforçar estas conclusões.

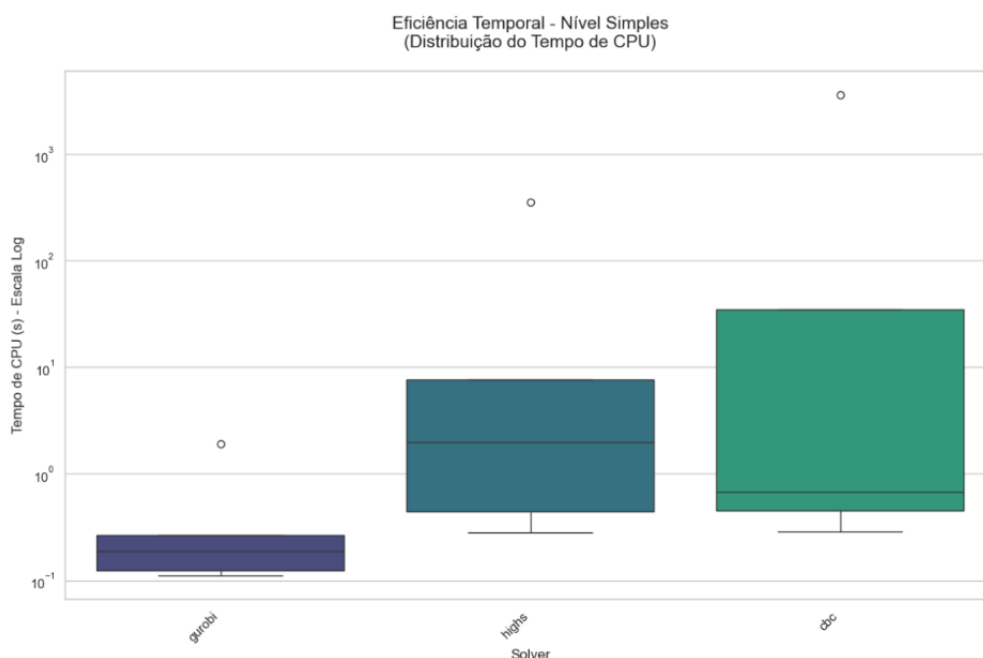


Figura 1- Eficiência Temporal das Instâncias Simples

4.2. INSTÂNCIAS DE NÍVEL INTERMÉDIO

Passando para as instâncias de nível intermédio, o cenário de análise é um pouco diferente. Embora a complexidade computacional permaneça significativa, estes problemas permitem que os *solvers* de código aberto demonstrem as suas capacidades de forma mais consistente, expondo diferenças de desempenho que não eram tão evidentes nos problemas mais simples. Os resultados detalhados são apresentados na Tabela seguinte:

Instância	Solver	Makespan Final	Tempo CPU (s)	Gap Relativo ao MOC (%)
InstM_01	Gurobi	189	9.5	0.00
	HiGHS	199	3600.0*	5.30
	CBC	215	3600.0*	13.80
InstM_02	Gurobi	192	3600.0*	0.00
	HiGHS	248	3600.0*	29.20
	CBC	264	3600.0*	37.50
InstM_03	Gurobi	179	3600.0*	0.00
	HiGHS	189	3600.0*	0.06
	CBC	259	3600.0*	44.70
InstM_04	Gurobi	224	3600.0*	0.00
	HiGHS	263	3600.0*	17.40
	CBC	320	3600.0*	42.90
InstM_05	Gurobi	193	3600.0*	0.00
	HiGHS	208	3600.0*	7.80
	CBC	280	3600.0*	45.10

Tabela 3 - Resultados detalhados para as instâncias de nível Intermédio

A análise da Tabela 6 demonstra uma superioridade do *Gurobi*, que estabeleceu o MOC em todas as cinco instâncias avaliadas

Para os solvers de código aberto, o cenário é de uma luta intensa contra a complexidade, onde ambos, *HiGHS* e *CBC*, esgotam o tempo máximo de execução de uma hora em praticamente todas as instâncias. Com o esforço computacional nivelado pelo tempo limite, o fator decisivo para a sua comparação passa a ser, exclusivamente, a qualidade do plano de trabalho final.

Neste confronto direto, o *HiGHS* revela-se uma ferramenta manifestamente superior. Consistentemente, entrega planos de trabalho mais eficientes, com um *Makespan* Final mais baixo e, conseqüentemente, um gap para a solução ótima muito menor. Em contraste, as

soluções do CBC, embora viáveis, resultam em operações mais longas e menos otimizadas, o que na prática se traduziria numa menor produtividade da oficina.

Métrica	Gurobi	HiGHS	CBC
Makespan Médio	195.4	221.4	267.6
Tempo CPU Médio (s)	2884.5	3600	3600
Gap Médio Relativo ao MOC (%)	0	13.1	36.8
Taxa de Sucesso - Atingiu MOC (%)	100	0	0

Tabela 4 - Desempenho médio dos solvers nas instâncias de nível Intermédio

Os dados médios da tabela 7 consolidam esta narrativa. Com um desvio médio de 13.1% face à melhor solução, o HiGHS demonstra ser uma ferramenta notavelmente mais eficaz que o CBC, cujo desvio médio atinge os 36.8%. Conclui-se, portanto, que para desafios de complexidade moderada, o HiGHS não só funciona de forma robusta, como representa a alternativa open-source com o melhor compromisso entre custo e performance, entregando resultados de qualidade significativamente superior.

4.3. INSTÂNCIAS DE NÍVEL COMPLEXO

A informação relativa à análise de desempenho para instâncias de nível complexo está presente na tabela seguinte:

Instância	Solver	Makespan Final	Tempo CPU (s)	Gap Relativo ao MOC (%)
InstC_01	Gurobi	378	80.1	0.00
	HiGHS	968	3600*	156.10
	CBC	545	3600*	44.20
InstC_02	Gurobi	445	3600*	0.00
	HiGHS	1157	3600*	160.00
	CBC	714	3600*	60.40
InstC_03	Gurobi	388	3600*	0.00
	HiGHS	N/A (Falhou)	3600*	-
	CBC	661	3600*	70.40
InstC_04	Gurobi	756	3600*	0.00
	HiGHS	3077	3600*	307.00
	CBC	1485	3600*	96.40
InstC_05	Gurobi	717	3600*	0.00
	HiGHS	N/A (Falhou)	3600*	-
	CBC	1949	3600*	171.80

Tabela 8 - Resultados detalhados para as instâncias de nível Complexo.

A análise da Tabela 8 revela um domínio por parte do solver Gurobi. Em todas as cinco instâncias, foi o Gurobi que estabeleceu o valor do MOC, demonstrando uma qualidade de solução inquestionavelmente superior. É de destacar o seu desempenho na instância Inst_06, que foi resolvida até à otimalidade em apenas 80 segundos, contrastando fortemente com os outros *solvers*, que esgotaram o tempo limite de uma hora.

O desempenho do HiGHS nestes cenários complexos expõe uma fragilidade crítica. Em duas das cinco, o solver não conseguiu encontrar uma única solução viável dentro do tempo limite, terminando o processo sem um resultado válido. Nos casos em que encontrou uma solução, o desempenho não foi muito bom, com desvios em relação ao MOC que variaram entre 156% e um valor extremo de 307%. Este comportamento sugere que, apesar do seu potencial, o HiGHS carece da robustez necessária para lidar de forma fiável com problemas desta magnitude.

O solver CBC, por sua vez, demonstrou ser mais estável que o HiGHS, ao conseguir encontrar sempre uma solução viável. Contudo, as soluções obtidas apresentam desvios significativos em relação ao MOC, que variaram entre 44% e um máximo de 172%.

A Tabela 5.2 resume estes resultados, apresentando os valores médios de desempenho para cada solver nas instâncias complexas.

Métrica	Gurobi	HiGHS	CBC
Makespan Médio	536.8	1734	1070.8
Tempo CPU Médio (s)	2901.1	3658.7	3602.3
Gap Médio Relativo ao MOC (%)	0	205	88.6
Taxa de Sucesso (Atingiu MOC)	1	0	0
Taxa de Falha - Não encontrou Sol. (%)	0	40	0

Tabela 9 - Desempenho médio dos solvers nas instâncias de nível Complexo.

4.4. CONCLUSÕES DA ANÁLISE EXPERIMENTAL

A análise de desempenho conduzida nos três níveis de complexidade das instâncias permitiu extrair um conjunto de conclusões robustas sobre a performance dos *solvers*. Ficou claramente demonstrado que, à medida que a complexidade do problema aumenta, a superioridade do *solver* comercial *Gurobi* se torna cada vez maior. A sua capacidade de encontrar consistentemente as melhores soluções, aliada a uma eficiência temporal que lhe permite resolver problemas em segundos ou minutos onde outros solvers requerem horas, posiciona-o como a referência indiscutível para problemas de otimização de grande escala.

Entre as alternativas de código aberto, o *HiGHS* emergiu como a opção mais promissora. Em problemas de complexidade simples e intermédia, demonstrou uma excelente qualidade de solução, superando claramente o *CBC*. No entanto, a sua performance nos cenários mais complexos revelou uma significativa falta de robustez, incluindo a incapacidade de encontrar soluções viáveis em parte dos casos testados. O *CBC*, em contrapartida, demonstrou maior consistência em não falhar a procura, ao contrário do *HiGHS* nos cenários mais complexos. Contudo, é fundamental sublinhar que a obtenção de uma solução viável não é sinónimo de uma solução de qualidade. À medida que a complexidade dos problemas aumentava, o tempo de computação limitado impedia o *CBC* de explorar eficazmente o espaço de soluções, resultando em planos de trabalho muito distantes do ótimo.

Face a estes resultados, e tendo em conta o objetivo de desenvolver um Sistema de Apoio à Decisão (SAD) eficaz e fiável, a decisão sobre a seleção do motor de otimização a ser integrado na solução final deve considerar o contexto e a disponibilidade de recursos da entidade implementadora. Se a empresa possuir a capacidade financeira para investir, a implementação com o *solver Gurobi* é fortemente aconselhada, uma vez que este demonstrou, de forma inequívoca, a combinação de robustez, qualidade da solução e eficiência necessárias para servir de base a uma aplicação industrial credível. No entanto, na ausência de disponibilidade financeira para adquirir uma licença comercial, será imperativo desenvolver uma abordagem que complemente o *solver HiGHS*. Embora esta ferramenta de código aberto apresente um potencial interessante, particularmente para problemas de menor dimensão, a sua eficácia em cenários mais complexos exigiria o desenvolvimento de heurísticas ou meta-heurísticas que auxiliassem na aceleração do processo de procura de uma primeira solução viável e na melhoria da qualidade das soluções obtidas. Desta forma, para o protótipo do Sistema de Apoio à Decisão a ser desenvolvido e apresentado no capítulo

seguinte, será utilizado o Gurobi, de modo a demonstrar a capacidade de otimização máxima que o modelo pode oferecer.

5. DESENVOLVIMENTO DO SISTEMA DE APOIO À DECISÃO E APLICAÇÃO A CASO DE ESTUDO

Concluída a análise comparativa de desempenho no capítulo anterior, que fundamentou a seleção de um motor de otimização de elevada performance, o presente trabalho avança para a sua fase de aplicação prática. Esta etapa consiste na materialização de toda a investigação efetuada, através do desenvolvimento de um protótipo funcional de um Sistema de Apoio à Decisão (SAD).

O objetivo deste capítulo é, portanto, descrever a conceção, a arquitetura de software e a aplicação desta ferramenta a um caso de estudo representativo do ambiente de uma oficina de reparação automóvel. Ao longo das próximas secções, será demonstrado como a modelação matemática e a análise experimental se traduzem numa solução com valor acrescentado e utilidade prática para o utilizador final, servindo como a ponte entre a teoria da otimização e a gestão operacional diária.

5.1. CARACTERIZAÇÃO DO CASO DE ESTUDO

Para validar a abordagem e a aplicação do modelo matemático, foi construído um caso de estudo representativo da realidade de uma oficina de reparação automóvel de média dimensão. O ponto de partida é o desafio diário enfrentado pelo gestor da oficina, ou planeador, que é a pessoa responsável por organizar o fluxo de trabalho. Este profissional enfrenta um ambiente complexo e dinâmico, onde a falta de uma ferramenta de apoio à decisão resulta frequentemente numa organização não ótima, baseada na experiência e na intuição, e com uma capacidade limitada para reagir a imprevistos.

O problema do planeador começa com a gestão da carga de trabalho para um determinado dia. O planeador dispõe de uma lista de viaturas que necessitam de intervenção, cada uma com um conjunto específico de tarefas a realizar, tempos de chegada e prazos de entrega. Ao mesmo tempo, tem de gerir um conjunto finito de recursos, tanto humanos (os mecânicos, com as suas diferentes especialidades e níveis de experiência) como físicos (os postos de trabalho, como elevadores, fossas e máquinas específicas). O seu objetivo é criar um plano de trabalho que aloque cada tarefa a um operador e a um posto, respeitando a sequência de operações de cada viatura, de forma a minimizar o tempo total e a maximizar a utilização dos recursos. Este processo, quando realizado manualmente, é propenso a ineficiências, levando a tempos de espera, recursos ociosos e, potencialmente, ao incumprimento de prazos.

Para formalizar este problema e permitir a sua resolução computacional, a informação dispersa do dia-a-dia da oficina foi estruturada e consolidada num único ficheiro de dados, em formato Excel. Este ficheiro é composto por um conjunto de folhas de cálculo interligadas que modelam a flexibilidade e as restrições do sistema, como a qualificação dos operadores para cada tipo de tarefa e a adequação dos postos de trabalho.

A análise quantitativa deste caso de estudo revela a sua dimensão e complexidade. O planeamento envolve um total de 20 viaturas e 41 operações distintas, a serem executadas por 7 operadores num conjunto de 12 postos de trabalho. A duração das operações é bastante heterogénea, variando entre um mínimo de 6 e um máximo de 144 unidades de tempo, com um valor médio de 20.8 e um desvio padrão elevado de 21.9, o que indica uma grande diversidade entre tarefas muito rápidas e outras muito longas. A dinâmica do problema é acentuada pelas janelas temporais das viaturas, cujos tempos de chegada se situam entre 0 e 48 e os prazos de entrega variam de 48 a 192, criando diferentes níveis de urgência. A

estrutura das tarefas é igualmente variada, com cada viatura a necessitar, em média, de 2.05 operações sequenciais.

Característica	Descrição Quantitativa
Dimensão Geral do Problema	
Número de Viaturas (Tarefas)	20
Número Total de Operações	41
Número de Operadores	7
Número de Postos de Trabalho	12
Estrutura das Tarefas	
Operações por Viatura	Média: 2.05 (Mín: 1, Máx: 4)
Duração das Operações (un. tempo)	Média: 20.8 (Mín: 6, Máx: 144)
Janelas Temporais (un. tempo)	
Intervalo de Chegada das Viaturas	[0, 48]
Intervalo de Prazos de Entrega	[48, 192]

5.2. ARQUITETURA E IMPLEMENTAÇÃO DO SAD

O Sistema de Apoio à Decisão foi desenvolvido em *Python*, seguindo uma arquitetura de *software* que separa as responsabilidades em dois componentes principais: um *backend* de modelação, responsável por toda a lógica de otimização, e um *frontend* de interação, que constitui a interface gráfica com o utilizador.

O *backend* da aplicação foi implementado num ficheiro *Python*. Este componente funciona como o "motor" do SAD, encapsulando toda a complexidade do problema. A sua primeira responsabilidade é a leitura e o processamento dos dados. Ele recebe o ficheiro Excel e interpreta as suas diversas folhas de cálculo, transformando a informação tabular numa estrutura de dados interna, composta por dicionários e listas *Python*, que é otimizada para a construção do modelo. Com base nesta estrutura, a sua segunda responsabilidade é construir dinamicamente o modelo de otimização MILP, traduzindo as tarefas, recursos e restrições para a formulação matemática descrita no Capítulo 3. Este processo é realizado com o auxílio da biblioteca PuLP, que permite criar as variáveis, definir as restrições e estabelecer a função objetivo de forma programática. O resultado do *backend* é um objeto *PuLP* completo.

O *frontend* da aplicação foi desenvolvido com recurso à biblioteca *Streamlit*. Esta biblioteca foi seleccionada por permitir a criação rápida de interfaces web interativas e esteticamente agradáveis, diretamente a partir de código Python. Este componente é totalmente responsável pela experiência do utilizador. Ele gere a interface gráfica, permitindo ao

planeador carregar o ficheiro Excel, definir parâmetros de execução como o tempo limite para a otimização e, com um simples clique num botão, acionar o processo.

O fluxo de informação entre os dois componentes é bem definido: o *frontend* invoca as funções do *backend*, passando os dados de entrada; o *backend* devolve o modelo matemático construído; o frontend submete então este modelo ao solver Gurobi para resolução; finalmente, o frontend recebe a solução e encarrega-se de a processar e formatar, utilizando bibliotecas como o Pandas para a manipulação de tabelas e o Plotly para a geração dos gráficos de Gantt interativos, que serão apresentados ao utilizador.

5.3. INTERFACE E FLUXO DE UTILIZAÇÃO DO SAD

A materialização do Sistema de Apoio à Decisão foi concretizada através do desenvolvimento de uma aplicação web interativa, cuja interface foi construída com recurso à biblioteca *Streamlit*. A filosofia de desenho da plataforma centrou-se na simplicidade e na criação de uma experiência de utilização intuitiva, adaptada aos diferentes papéis e necessidades informacionais existentes no ambiente de uma oficina. Para evitar a sobrecarga de informação e maximizar a pertinência dos dados apresentados, o fluxo de interação da aplicação é segmentado por perfis de utilizador, garantindo que cada elemento da equipa acede apenas às funcionalidades e visualizações que lhe são diretamente relevantes.

5.3.1. ECRÃ DE ENTRADA E SELEÇÃO DE PERFIL

Ao aceder à plataforma, o utilizador é recebido por um ecrã inicial que funciona como um portal de entrada para o sistema como é visível na figura 2. Neste ecrã, é apresentado um menu de seleção que permite a escolha de um de três tipos de perfil: Planeador, operador ou posto de trabalho. Uma característica fundamental do sistema, que lhe confere uma elevada adaptabilidade, é que as listas de operadores e postos de trabalho não são fixas, são criadas dinamicamente a partir dos dados carregados no ficheiro Excel de entrada, ou de um ficheiro de dados padrão no arranque inicial da aplicação. Isto significa que a plataforma se ajusta automaticamente à equipa e à configuração física de qualquer oficina, demonstrando a sua flexibilidade e aplicabilidade a diferentes contextos operacionais sem necessidade de reconfiguração do código-fonte.

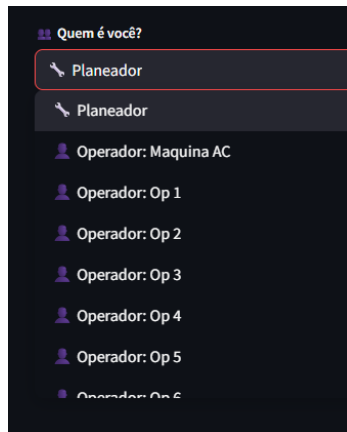


Figura 2 - Menu de entrada

5.3.2. A VIDA DO PLANEADOR: O CENTRO DE COMANDO

A vista do Planeador constitui o centro do sistema, onde toda a otimização é configurada, executada e, subsequentemente, analisada. A interface foi desenhada de forma lógica, com uma barra lateral dedicada às configurações do planeamento e uma área principal destinada à visualização dos resultados. O fluxo de trabalho do planeador é intuitivo e sequencial, guiando o utilizador através de três passos claros.

O primeiro passo consiste em Carregar os Dados do Dia, onde o utilizador pode submeter um ficheiro Excel com a carga de trabalho ou optar por utilizar um ficheiro padrão. Para garantir a performance e a reatividade da aplicação, foi implementado um sistema de cache que armazena em memória os dados processados, evitando o seu reprocessamento desnecessário em utilizações subsequentes do mesmo ficheiro. O segundo passo é a definição do tempo limite de processamento, em segundos. Esse valor será fornecido como parâmetro ao SAD e representa o tempo que o *solver* terá para encontrar a solução, permitindo assim um compromisso ajustável entre a qualidade da solução e o tempo de computação. Finalmente, com um clique no botão Gerar Plano de Trabalho, o processo de otimização é iniciado. A interface mantém o utilizador informado do progresso e, no final, apresenta a solução de forma detalhada no *dashboard* principal como está seguidamente representada



Figura 3- Vista do Planeador com resultados

Após a conclusão do processo, a área principal é preenchida com um dashboard completo. No topo, são exibidos os Indicadores Chave de Desempenho que fornecem uma visão sumária e imediata da eficiência do plano gerado. No caso de estudo, estes indicadores revelaram um *makespan* de 122.40 unidades de tempo, uma utilização média dos operadores de 119.3% e uma utilização média dos postos de 59.6%. O valor da utilização dos operadores, superior a 100%, constitui uma informação de gestão de enorme valor, indicando que a solução ótima para a carga de trabalho do dia requer a alocação de horas no dia seguinte, uma realidade que a ferramenta quantifica de forma explícita.

O núcleo da visualização de resultados reside nos gráficos de *Gantt* interativos, gerados com a biblioteca Plotly. A plataforma gera automaticamente dois gráficos distintos, organizados em separadores: um que apresenta o plano de trabalho por operador como está representado na figura 4 e outro por posto de trabalho como é visível na figura 5. Estes gráficos permitem ao planeador obter detalhes sobre cada tarefa, de forma crucial, visualizar uma linha vertical vermelha que marca o Fim do Dia (correspondente a 96 unidades de tempo, ou 8 horas). Esta funcionalidade permite identificar de imediato quais as tarefas que se estendem para além do horário normal de trabalho.

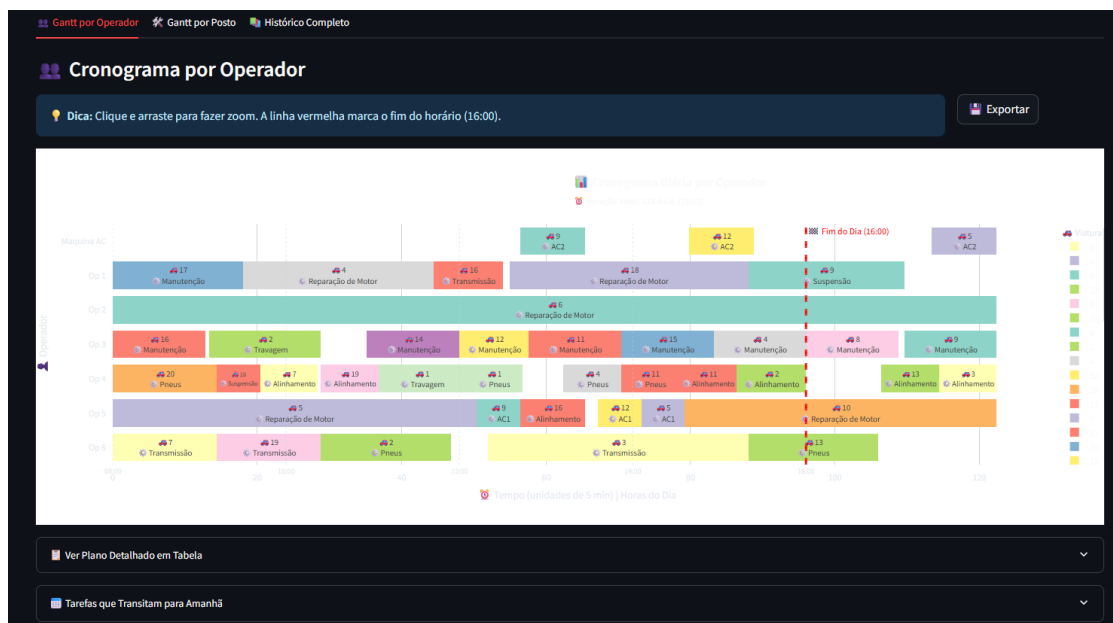


Figura 4 - Gantt por Operador

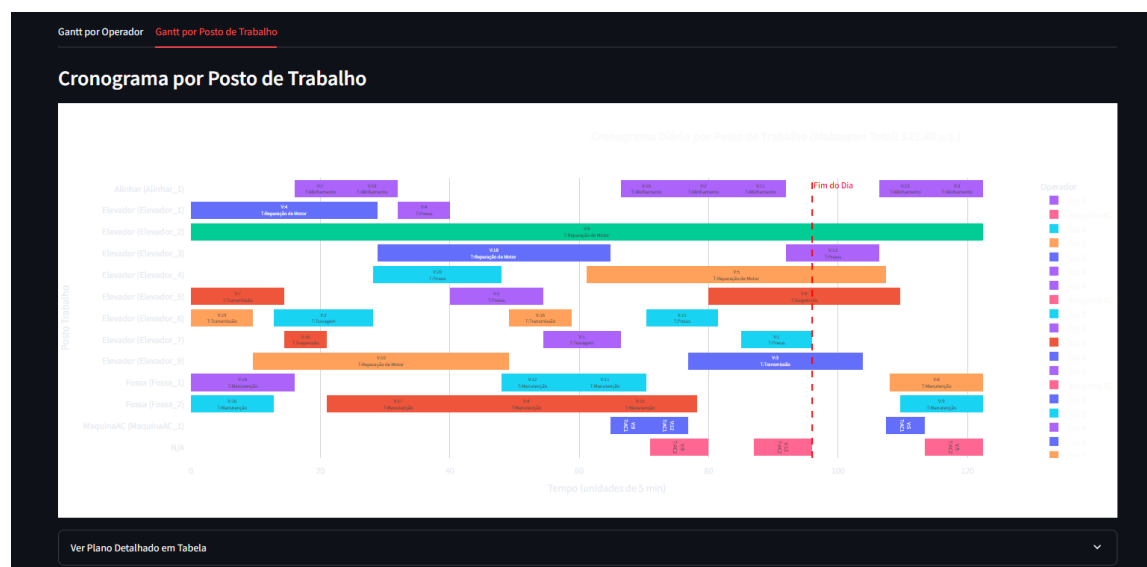


Figura 5- Gantt por Posto de Trabalho

Para uma análise mais profunda, a plataforma disponibiliza ainda uma tabela com o plano completo. De forma a apoiar o planeamento contínuo, uma secção dedicada às “Tarefas que Transitarão para o Próximo Dia” lista todas as operações não concluídas, juntamente com uma estimativa do tempo restante como está por baixo representado. Desta forma, o *dashboard* do Planeador materializa os ganhos da otimização, convertendo um problema de

escalonamento complexo numa solução visual e acionável que capacita o gestor com as informações quantitativas necessárias para uma gestão proativa e estratégica da oficina.



	ID Tarefa	Viatura	Tipo Tarefa	Tempo Restante Estimado
0	6	3	Transmissão	7.8
1	7	3	Alinhamento	8
2	11	5	Reparação de Motor	11.4
3	12	5	AC1	6
4	13	5	AC2	9
5	14	6	Reparação de Motor	26.4
6	17	8	Manutenção	14.4
7	18	9	Suspensão	13.6
8	19	9	Manutenção	12.8
9	29	13	Pneus	10.4

Figura 6 - Tarefas que transitam para o dia seguinte

5.3.3. AS VISTAS DO OPERADOR E DO POSTO DE TRABALHO: AGENDAS DIÁRIAS FOCADAS

Uma vez gerado o plano pelo gestor, os restantes utilizadores podem aceder à plataforma e seleccionar o seu perfil específico para obter uma visão simplificada e direccionada do seu dia de trabalho.

Ao seleccionar um perfil de Operador, o sistema apresenta uma agenda diária personalizada como está representado na figura 7. Esta lista, de forma cronológica, apenas as tarefas atribuídas a esse mecânico, detalhando os horários de início e fim, a viatura, o tipo de tarefa e o posto de trabalho a utilizar. Adicionalmente, é apresentada a taxa de ocupação individual para o dia. Para lidar com cenários de sobrecarga, a barra de progresso visual foi limitada a 100%, enquanto a métrica textual exibe o valor real, informando o operador de forma transparente sobre a sua carga de trabalho sem causar erros na interface.

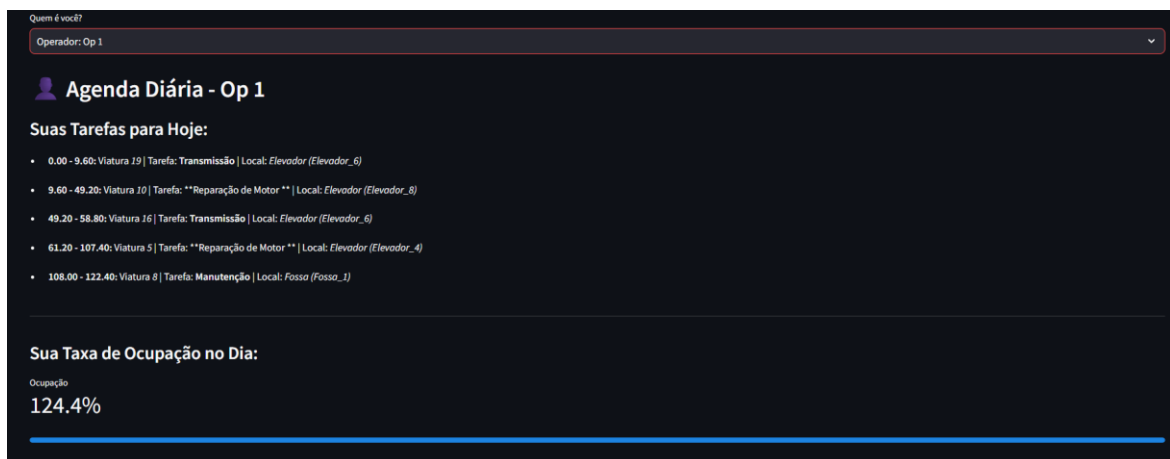


Figura 7 - Agenda operador

A vista de Posto de Trabalho (figura 8) mostra o cronograma de utilização de um recurso físico específico, como um elevador ou uma fossa, detalhando que tarefas, com que viaturas e operadores, irão ocupar esse posto ao longo do dia. Estas vistas personalizadas são fundamentais para melhorar a comunicação e a coordenação na oficina, funcionando como uma fonte única e fidedigna de informação que garante que todos os elementos da equipa estão sincronizados, o que resulta na redução de tempos mortos e na execução fluida do plano de trabalho otimizado.

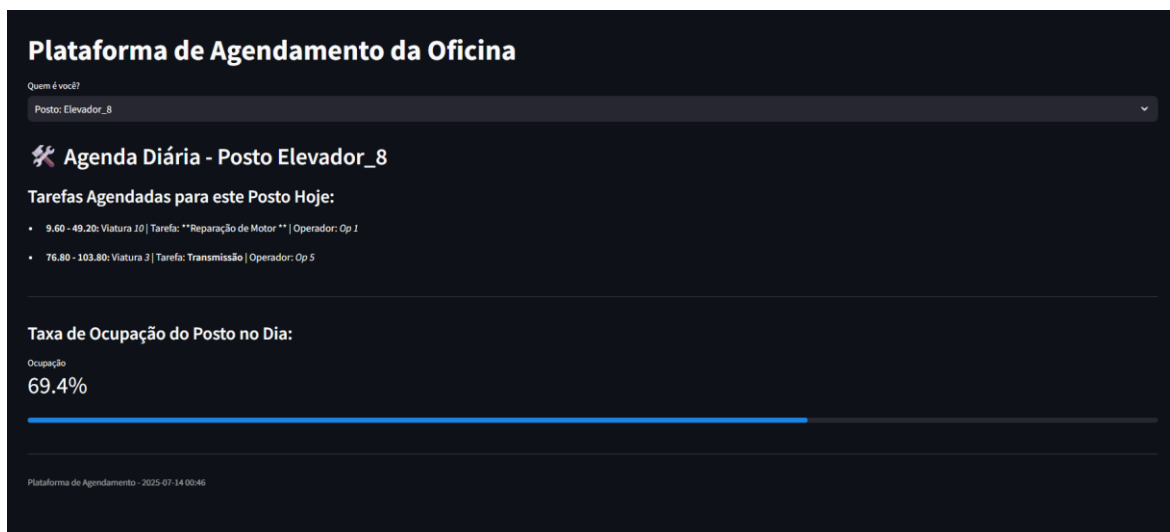


Figura 8 - Agenda posto de trabalho

5.4. IMPACTO E CONTRIBUTOS DO SISTEMA APOIO À DECISÃO

A aplicação do protótipo ao caso de estudo permitiu não só validar a robustez do modelo matemático, mas, acima de tudo, demonstrar o valor prático e o potencial transformador da ferramenta. A solução desenvolvida transcende a mera automação de tarefas, representando uma mudança de paradigma na gestão da oficina, cujos principais contributos se detalham de seguida.

O primeiro e mais significativo contributo é a transição de um planeamento empírico para uma tomada de decisão baseada em dados. Ao substituir a intuição do gestor por uma solução matematicamente otimizada, o SAD garante a máxima eficiência teórica na utilização dos recursos, minimizando tempos ociosos e maximizando o volume de produção.

Em segundo lugar, a ferramenta gera conclusões de gestão que seriam extremamente difíceis de obter manualmente. Um exemplo claro é a quantificação exata da necessidade de trabalho suplementar, evidenciada pela taxa de utilização dos operadores superior a 100%. Esta métrica fornece ao gestor uma justificação sólida e quantificável para alocar horas extraordinárias ou para renegociar prazos de forma proativa.

Finalmente, a plataforma prova o seu valor como um assistente de planeamento contínuo e dinâmico. A identificação automática das tarefas que transitam para o dia seguinte e a apresentação de agendas personalizadas para cada colaborador melhoram drasticamente a comunicação, a coordenação e a agilidade da equipa para reagir a imprevistos.

Para concluir, o protótipo desenvolvido cumpre com sucesso os objetivos do projeto, materializando-se numa solução de gestão poderosa. Demonstra ser capaz de gerar um impacto positivo e mensurável na eficiência, produtividade e, consequentemente, na rentabilidade de uma oficina de reparação automóvel.

6. CONCLUSÕES E TRABALHO FUTURO

O presente trabalho abordou o desafio da otimização do escalonamento de operações em oficinas de reparação automóvel, um problema de elevada complexidade teórica e de manifesta relevância para o tecido empresarial nacional, dominado por pequenas e médias empresas com acesso limitado a soluções tecnológicas avançadas. O objetivo central foi, assim, o de desenvolver e validar um Sistema de Apoio à Decisão eficaz e funcional, que demonstrasse a viabilidade de aplicar a investigação operacional a este contexto. Para tal, foi adotada uma metodologia rigorosa que se iniciou com a formulação de um modelo de Programação Linear Inteira Mista (*MILP*) e culminou no desenvolvimento de um protótipo interativo.

Os objetivos delineados foram plenamente atingidos. O projeto demonstrou que é viável criar uma ferramenta de alto desempenho que responde a uma necessidade real do mercado, combinando o rigor da modelação matemática com a acessibilidade de tecnologias de *software* modernas. O protótipo desenvolvido não se limita a gerar um plano de trabalho otimizado; oferece conclusões de gestão valiosas, como a quantificação precisa da necessidade de horas extraordinárias, transformando-se num verdadeiro assistente ao planeamento estratégico e diário da oficina.

Apesar do sucesso alcançado, o presente trabalho possui limitações que abrem caminho para futuras investigações. O modelo proposto é de natureza determinística, não contemplando a incerteza inerente a uma oficina, como avarias inesperadas ou variações nos tempos das tarefas. Nesse sentido, a investigação futura poderia seguir várias vertentes complementares. Primeiramente, no âmbito da validação experimental, seria pertinente alargar o conjunto de testes com um maior número de instâncias e avaliar o comportamento do modelo em horizontes temporais mais vastos, como uma semana completa de planeamento, para aferir a sua escalabilidade. Do ponto de vista da otimização, seria de grande valor a implementação

de outras funções-objetivo que se adaptem a realidades empresariais específicas, como a minimização de custos ou de atrasos, bem como a exploração de métodos híbridos que auxiliem os *solvers open-source* a acelerar a obtenção de uma primeira solução viável e a melhorar a sua qualidade. Finalmente, o protótipo poderia ser funcionalmente enriquecido com módulos de gestão de inventário de peças ou integração com sistemas de faturação, sendo que a sua validação em ambiente real de oficina constituiria o passo decisivo para comprovar o seu impacto.

A um nível pessoal, a realização deste projeto no *INESC TEC* revelou-se uma experiência de enorme valor formativo. A oportunidade de trabalhar num ambiente de investigação de excelência permitiu-me não só aplicar os conhecimentos teóricos adquiridos ao longo da licenciatura a um problema industrial concreto, mas também desenvolver um conjunto de competências técnicas e transversais. A necessidade de modelar um problema complexo, de desenhar e executar uma metodologia experimental robusta, e de desenvolver uma solução de *software* funcional aprofundou as minhas capacidades nas áreas da otimização e da análise de dados, enquanto a gestão autónoma do projeto foi fundamental para o desenvolvimento de competências de resolução de problemas e pensamento crítico.

Para finalizar, este trabalho representa não apenas a concretização dos objetivos académicos propostos, mas também uma contribuição prática com potencial para melhorar a eficiência e a competitividade no setor da manutenção automóvel, constituindo um culminar gratificante do meu percurso de licenciatura e da valiosa experiência de estágio.

Referências Bibliográficas

- [1] M. Mailliez et al., “Production scheduling and rescheduling with decision support: A human factors perspective,” *Frontiers in Neuroergonomics*, vol. 2, 2021.
- [2] S. Lee, “Tackling Scheduling Challenges with Combinatorial Optimization,” *NumberAnalytics Blog*, May 2024.
- [3] X. Kong, D. Han, and J. Sun, “Research on Production Scheduling of Flexible Job Shop Based on Improved NSGA-II Algorithm,” *Processes*, vol. 12, no. 1, p. 195, Jan. 2024.
- [4] M. L. Pinedo, *Scheduling: Theory, Algorithms, and Systems*, 6th ed. Springer, 2022.
- [5] R. L. Graham, E. L. Lawler, J. K. Lenstra, and A. H. G. Rinnooy Kan, “Optimization and approximation in deterministic sequencing and scheduling: a survey,” *Annals of Discrete Mathematics*, vol. 5, pp. 287–326, 1979.
- [6] J. F. C. Applegate, D. L., & Bixby, R. E., Cook, W. J., & Chvátal, V. (2007). *The traveling salesman problem: A computational study*. Princeton university press.
- [7] X. Kong, S. Liu, C. Gao, and L. Gao, “Solving the Flexible Job Shop Scheduling Problem Using a Discrete Improved Grey Wolf Optimization Algorithm,” *Machines*, vol. 10, no. 11, p. 1045, Nov. 2022.
- [8] F. Deng and J. Chen, “End-to-End Deep Reinforcement Learning for Solving the Flexible Job-Shop Scheduling Problem,” *arXiv preprint arXiv:2305.05119*, 2023.
- [9] Gurobi Optimization, LLC, “Mixed-Integer Programming (MIP) – A Primer on the Basics,” 2024. [Online].
- [10] G. L. Nemhauser and L. A. Wolsey, *Integer and Combinatorial Optimization*. John Wiley & Sons, 1999.
- [11] R. Ruiz and J. C. Framiñan, “A Review of Heuristics for Shop Scheduling,” in *Handbook of Heuristics*, R. Martí, P. M. Pardalos, and M. G. C. Resende, Eds. Springer, 2018, pp. 1-27.
- [12] Gurobi Optimization, LLC, “Gurobi Optimizer Reference Manual,” 2024. [Online].
- [13] COIN-OR Foundation, “Cbc (Coin-or branch and cut) User Guide,” [Online].
- [14] J. A. J. Hall, Q. Huangfu, and I. Papatsoumara, “HiGHS - High-performance open-source software for linear optimization,” [Online].
- [15] A. Makhorin, “GLPK (GNU Linear Programming Kit),” GNU Project, [Online].
- [16] Wikipedia, “Job-shop scheduling,” *en.wikipedia.org*, 2024. [Online]

- [17] A. Zhang *et al.*, “On the NP-hardness of scheduling with time restrictions”, *arXiv preprint arXiv:1710.06732*, 2017.
- [18] J. P. Vielma, “Mixed Integer Linear Programming Formulation Techniques”, *SIAM Review*, vol. 57, no. 1, pp. 3–57, 2015.
- [19] Optimization Wiki, “Disjunctive inequalities,” *optimization.mccormick.northwestern.edu*, 2021. [Online].
- [20] H. Mittelmann, *Benchmarking Optimization Software*, 2024. [Online].
- [21] J. Hall *et al.*, "Optimization solvers: the missing link for a fully open-source energy," *HiGHS funding proposal*, 2022.
- [22] Z. Ye *et al.*, “Comparison of open source solvers HiGHS, GLPK and Cbc to commercial solver Gurobi,” *ResearchGate*, 2022. [Online].
- [23] SciComp StackExchange, “What’s the fastest software ... to solve mixed integer programming?”, 2012. [Online]. Available: